TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

UTN
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

PROGRAMACIÓN II

Trabajo Práctico 5: Relaciones UML 1 a 1

OBJETIVO GENERAL

Modelar clases con relaciones 1 a 1 utilizando diagramas UML. Identificar correctamente el tipo de relación (asociación, agregación, composición, dependencia) y su dirección, y llevarlas a implementación en Java.

Caso Práctico

Desarrollar los siguientes ejercicios en Java. Cada uno deberá incluir:

- ✓ ● Diagrama UML
- ✓ ● Tipo de relación (asociación, agregación, composición, dependencia)
- ✓ ● Dirección (unidireccional o bidireccional)
- ✓ ● Implementación de las clases con atributos y relaciones definidas

Link de GitHub:
https://github.com/Cigarcia1307/Programacion2-UTN-GarciaCintia.git

Ejercicios de Relaciones 1 a 1

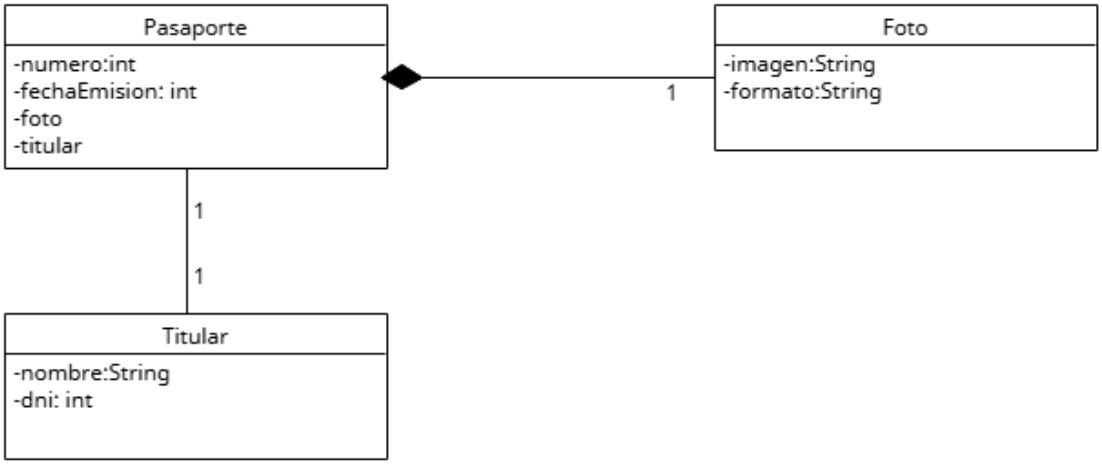
1. Pasaporte - Foto - Titular

a. Composición: Pasaporte → Foto
b. Asociación bidireccional: Pasaporte ↔ Titular

Clases y atributos:

i. Pasaporte: numero, fechaEmision
ii. Foto: imagen, formato
iii. Titular: nombre, dni

UML:




EjercicioRelacion1.java [-/A] × | Pasaporte.java [-/A] × | Titular.java [-/A] × | Foto.java [-/A] ×

Source | History

```java
*/
public class Pasaporte {
    private int numero;
    private int fechaEmision;
    private Foto foto;//composicion lal
    private Titular titular;//asociacion bidireccional

    public Pasaporte(int numero, int fechaEmision, String imagen, String formato) {
        this.numero = numero;
        this.fechaEmision = fechaEmision;
        this.foto = new Foto (imagen, formato);

    }

    public void mostrarFoto(){
        System.out.println("El pasaporte: " + numero + " fecha de emision " + fechaEmision + " tiene la im

    }

    public Pasaporte(Titular titular) {
        this.titular = titular;
    }


    public void setTitular(Titular titular) {
        this.titular = titular;
    }
```

EjercicioRelacion1.java [-/A] × | Pasaporte.java [-/A] × | Titular.java [-/A] × | Foto.java [-/A] ×

```java
*/
public class Titular {
    private String nombre;
    private int dni;
    private Pasaporte pasaporte; //referencia al pasaporte asociado

    public Titular(String nombre, int dni, Pasaporte pasaporte) {
        this.nombre = nombre;
        this.dni = dni;
        this.pasaporte = pasaporte;
        pasaporte.setTitular(this);
    }


    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getDni() {
        return dni;
    }

    public void setDni(int dni) {
        this.dni = dni;
    }

    public Pasaporte getPasaporte() {
        return pasaporte;
```

EjercicioRelacion1.java [-/A] × | Pasaporte.java [-/A] × | Titular.java [-/A] × | Foto.java [-/A] ×

```java
 * @author HP
 */
public class Foto {
    private String imagen;
    private String formato;

    public Foto(String imagen, String formato) {
        this.imagen = imagen;
        this.formato = formato;
    }

    public String getImagen() {
        return imagen;
    }

    public String getFormato() {
        return formato;
    }
```

tput ×

HP - C:\Users\HP × | EjercicioRelacion1 (run) ×

```
run:
El pasaporte: 1234 fecha de emision 14 tiene la imagen  foto1.jpg  formato  jpeg
BUILD SUCCESSFUL (total time: 0 seconds)
```

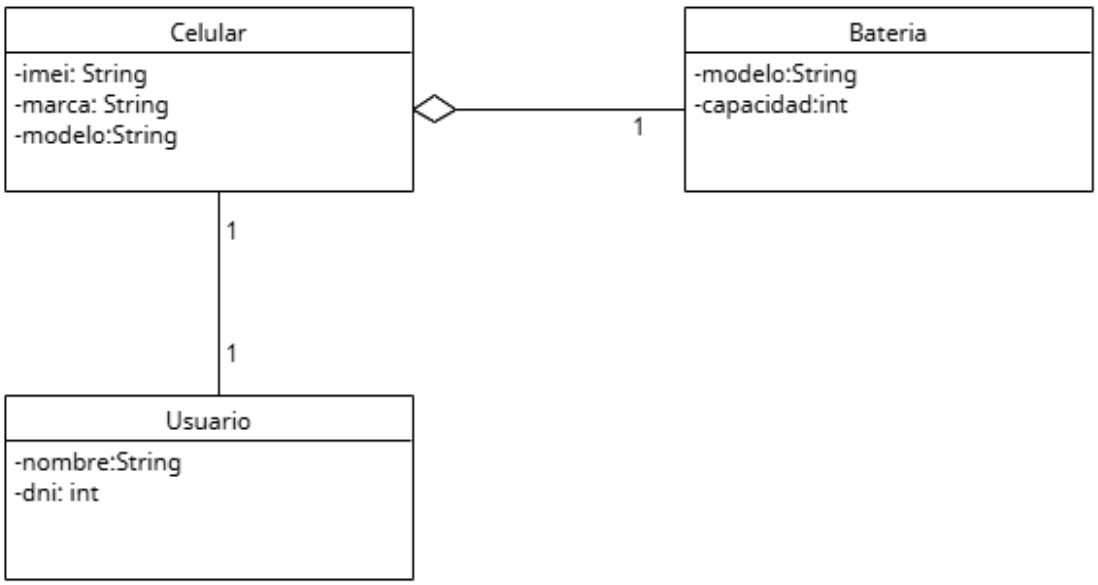
2. Celular - Batería - Usuario

a. Agregación: Celular → Batería
b. Asociación bidireccional: Celular ↔ Usuario

Clases y atributos:

i. Celular: imei, marca, modelo
ii. Batería: modelo, capacidad
iii. Usuario: nombre, dni

UML:



EjercicioRelacion2.java [-/A] × | Celular.java [-/A] × | Bateria.java [-/A] × | Usuario.java [-/A] ×

ource History

```java
*/
public class Usuario {
    private String nombre;
    private int dni;
    private Celular celular; //referencia al celular asociado

    public Usuario(String nombre, int dni) {
        this.nombre = nombre;
        this.dni = dni;

    }
    public void setCelular (Celular celular){
        this.celular =celular;
        if(celular != null && celular.getUsuario() != this){
            celular.setUsuario (this);
        }
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getDni() {
        return dni;
```

EjercicioRelacion2.java [-/A] × | Celular.java [-/A] × | Bateria.java [-/A] × | Usuario.java [-/A] ×

Source | History

```java
package ejerciciorelacion2;

/**
 *
 * @author HP
 */
public class Bateria {
    private String modelo;
    private int capacidad;

    public Bateria(String modelo, int capacidad) {
        this.modelo = modelo;
        this.capacidad = capacidad;
    }

    public String getModelo() {
        return modelo;
    }



    public int getCapacidad() {
        return capacidad;
    }

```

```java
*/
public class Celular {
    private String imei;
    private String marca;
    private String modelo;
    private Bateria bateria;//agregacion lal
    private Usuario usuario;//asociacion bidireccional

    public Celular(String imei, String marca, String modelo, Bateria bateria
        this.imei = imei;
        this.marca = marca;
        this.modelo = modelo;
        this.bateria = bateria;
    }

    public void setBateria(Bateria bateria){
        this.bateria =bateria;

    }
    public String getMarca() {
        return marca;
    }

    public String getModelo() {
        return modelo;
    }

    public Usuario getUsuario() {
```

```java
public class EjercicioRelacion2 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Bateria bateria1 = new Bateria("Bater", 100);
        Celular celular1= new Celular("012345874", "Iphone", "13ProMax",bateria1);
        Usuario usuario= new Usuario("Rafael Perez", 35478956);

        celular1.setUsuario(usuario);

        celular1.mostrarInfo();
        System.out.println("Usuario asociado: " + usuario.getNombre() + ", DNI: " + usuario.getDni());
        System.out.println("Celular del usuario: " + usuario.getCelular().getMarca() + " " + usuario.getCelu
```

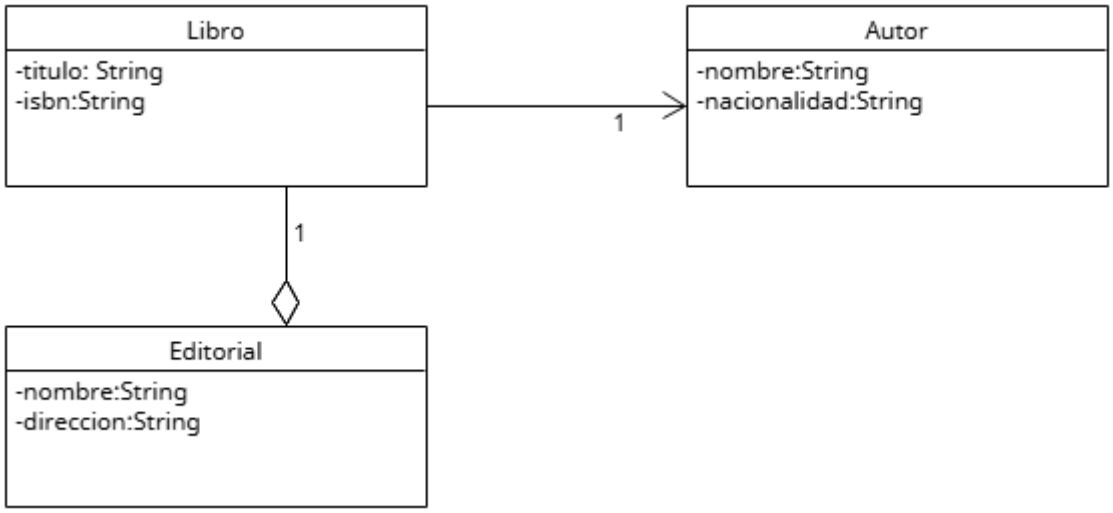
```
run:
Informacion del Celular:
  IMEI: 012345874
  Marca: Iphone
  Modelo: 13ProMax
  Bateria: Bater con 100 porcentaje
Usuario asociado: Rafael Perez, DNI: 35478956
Celular del usuario: Iphone 13ProMax
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Libro - Autor - Editorial

a. Asociación unidireccional: Libro → Autor
b. Agregación: Libro → Editorial

Clases y atributos:

i. Libro: titulo, isbn
ii. Autor: nombre, nacionalidad
iii. Editorial: nombre, direccion



EjercicioRelacion3.java [-/A] × | Libro.java [-/A] × | Editorial.java [-/A] × | Autor.java [-/A] ×

Source | History

```java
 */
package ejerciciorelacion3;

/**
 *
 * @author HP
 */
public class Autor {
    private String nombre;
    private String nacionalidad;

    public Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }

    public String getNombre() {
        return nombre;
    }

    public String getNacionalidad() {
        return nacionalidad;
    }
```

```java
/**
 *
 * @author HP
 */
public class Editorial {
    private String nombre;
    private String direccion;

    public Editorial(String nombre, String direccion) {
        this.nombre = nombre;
        this.direccion = direccion;
    }

    public String getNombre() {
        return nombre;
    }

    public String getDireccion() {
        return direccion;
    }
```

```java
 * @author HP
 */
public class Libro {
    private String titulo;
    private String isbn;
    private Autor autor; //asociacion 1 a 1
    private Editorial editorial;//agregacion

    public Libro(String titulo, String isbn) {
        this.titulo = titulo;
        this.isbn = isbn;
    }

    public Editorial getEditorial() {
        return editorial;
    }

    public void setEditorial(Editorial editorial) {
        this.editorial = editorial;
    }

    public void setAutor (Autor autor){
        this.autor = autor;

    }
    public void mostrarAutor(){
        if(autor != null){
            System.out.println(titulo +  isbn + "el autor es:" + autor.getNombre() + ",de nacionalidad " +
```

```java
 * @author HP
 */
public class EjercicioRelacion3 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Editorial editorial =new Editorial (" Suma ", "Suipacha 123");
        Autor autor = new Autor(" Florencia Bonelli ", " Argentina");
        Libro libro=new Libro ("Caballos de fuego ", " A1024 ");

        libro.setAutor(autor);
        libro.setEditorial(editorial);

        libro.mostrarAutor();
    }

}
```

```
run:
Caballos de fuego  A1024 el autor es: Florencia Bonelli ,de nacionalidad  Argentina
Editorial:  Suma  (Suipacha 123)
BUILD SUCCESSFUL (total time: 0 seconds)
```

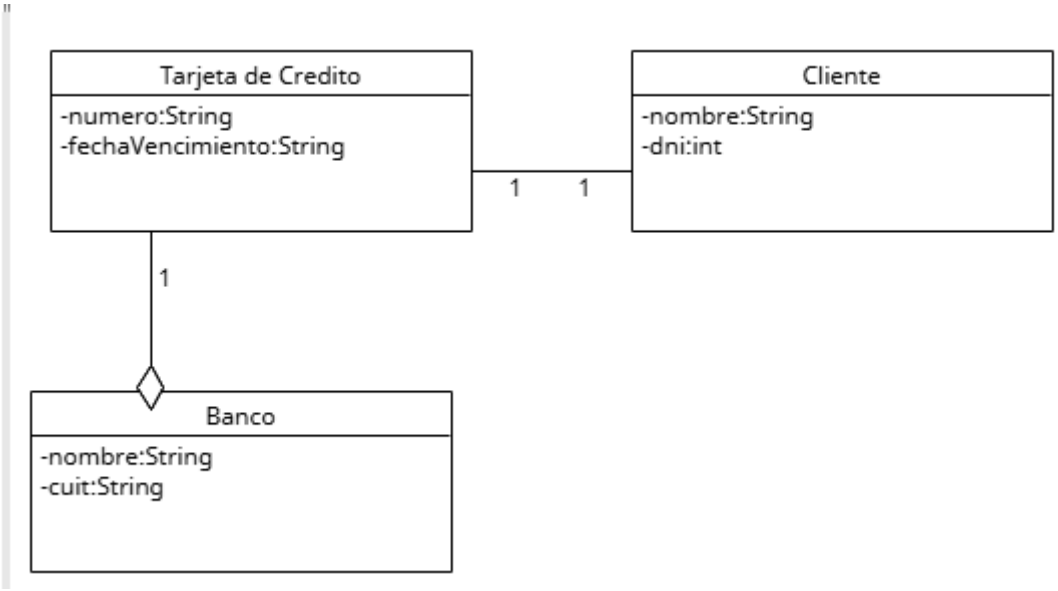
4. TarjetaDeCrédito - Cliente - Banco

a. Asociación bidireccional: TarjetaDeCrédito ↔ Cliente

b. Agregación: TarjetaDeCrédito → Banco

Clases y atributos:

i. TarjetaDeCrédito: numero, fechaVencimiento
ii. Cliente: nombre, dni
iii. Banco: nombre, cuit



EjercicioRelacion4.java [-/A] × | TarjetaCredito.java [-/A] × | Banco.java [-/A] × | Cliente.java [-/A] ×

Source | History

```java
/**
 *
 * @author HP
 */
public class EjercicioRelacion4 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        TarjetaCredito tarjetaCredito = new TarjetaCredito("459652145287", "10/20");
        Banco banco= new Banco ("Santander", "2005874521");

        tarjetaCredito.setBanco(banco);
        tarjetaCredito.mostrarInfo();
    }

}
```

Output ×

HP - C:\Users\HP × | EjercicioRelacion4 (run) ×

```
run:
Numero de tarjeta: 459652145287
Fecha de vencimiento: 10/20
Banco: Santander - CUIT: 2005874521
BUILD SUCCESSFUL (total time: 0 seconds)
```

EjercicioRelacion4.java [-/A] × | TarjetaCredito.java [-/A] × | Banco.java [-/A] × | Cliente.java [-/A] ×

Source | History

```java
 *
 * @author HP
 */
public class TarjetaCredito {
    private String numero;
    private String fechaVencimiento;
    private Cliente cliente; // para la asociación bidireccional
    private Banco banco;     // para la agregación


    public TarjetaCredito(String numero, String fechaVencimiento) {
        this.numero = numero;
        this.fechaVencimiento = fechaVencimiento;
    }

    public void setCliente(Cliente cliente){
        this.cliente =cliente;
        if (cliente != null && cliente.getTarjetaCredito() != this){
            cliente.setTarjetaCredito (this);
        }

    }

    public Cliente getCliente() {
        return cliente;
    }

    public void setBanco(Banco banco) {
```

Output ×

EjercicioRelacion4.java [-/A] × | TarjetaCredito.java [-/A] × | Banco.java [-/A] × | Cliente.java [-/A

Source | History

```java
/**
 *
 * @author HP
 */
public class Banco {
    private String nombre;
    private String cuit;

    public Banco(String nombre, String cuit) {
        this.nombre = nombre;
        this.cuit = cuit;
    }

    public String getNombre() {
        return nombre;
    }

    public String getCuit() {
        return cuit;
    }
```

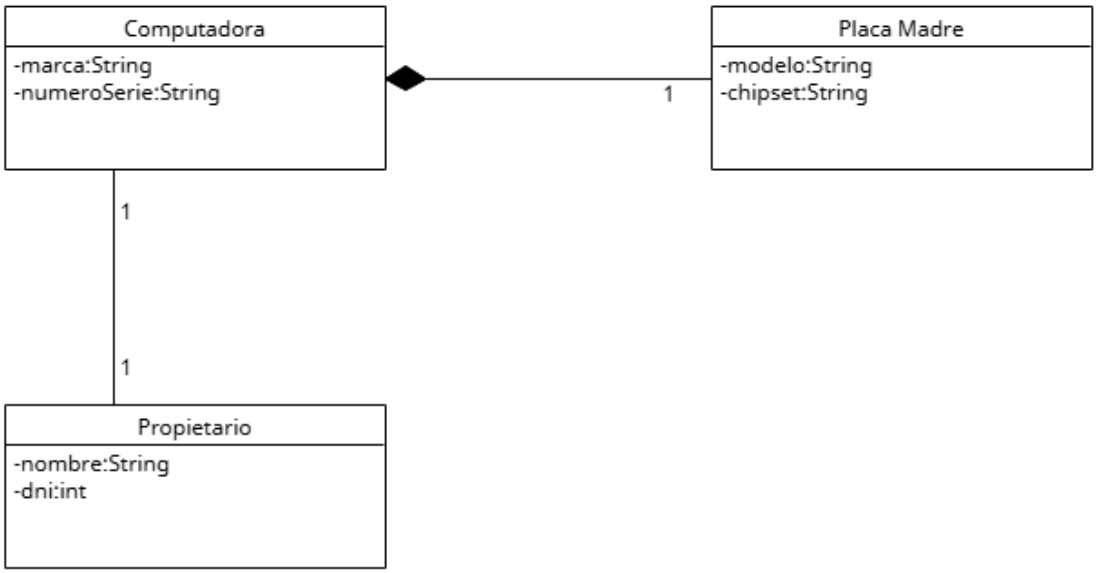
```java
*/
public class Cliente {
    private String nombre;
    private int dni;
    private TarjetaCredito tarjetaCredito;

    public Cliente(String nombre, int dni) {
        this.nombre = nombre;
        this.dni = dni;
    }

    public void setTarjetaCredito(TarjetaCredito tarjetaCredito) {
        this.tarjetaCredito = tarjetaCredito;
    }

    public TarjetaCredito getTarjetaCredito() {
        return tarjetaCredito;
    }

    public String getNombre() {
        return nombre;
    }

    public int getDni() {
        return dni;
    }
```

5. Computadora - PlacaMadre - Propietario

a. Composición: Computadora → PlacaMadre
b. Asociación bidireccional: Computadora ↔ Propietario

Clases y atributos:

i. Computadora: marca, numeroSerie
ii. PlacaMadre: modelo, chipset
iii. Propietario: nombre, dni




EjercicioRelacion5.java [-/A] × | Computadora.java [-/A] × | PlacaMadre.java [-/A] × | Propietario.java [-/A] ×

Source | History

```java
 */
public class EjercicioRelacion5 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Computadora notebook =new Computadora ("HP ", "10244254758","HP5847415", "1254MP");
        Propietario propietario = new Propietario("Alejandro Fernandez", "26065874");
        notebook.mostrarPlacaMadre();
        notebook.setPropietario(propietario);
        notebook.mostrarPropietario();

    }

}
```

Output ×

HP - C:\Users\HP × | EjercicioRelacion5 (run) ×

```
run:
La computadora: HP  numero de serie :10244254758 pertenece al modelo SERIE:HP5847415 y numero de chipset:1254MP
Popietario: Alejandro Fernandeznumero DNI:26065874
BUILD SUCCESSFUL (total time: 0 seconds)
```

EjercicioRelacion5.java [-/A] × | Computadora.java [-/A] × | PlacaMadre.java [-/A] × | Propietario.java [-/A] ×

Source | History

```java
/**
 *
 * @author HP
 */
public class Computadora {
    private String marca;
    private String numeroSerie;
    private PlacaMadre placaMadre; //composicion 1a1
    private Propietario propietario;//asociacion bidireccional

    public Computadora(String marca, String numeroSerie, String modelo, String chipset) {
        this.marca = marca;
        this.numeroSerie = numeroSerie;
        this.placaMadre = new PlacaMadre(modelo, chipset);
    }

    public void mostrarPlacaMadre(){
        System.out.println("La computadora: " + marca + " numero de serie :" + numeroSerie + " pertenece a
    }

    public void setPropietario (Propietario propietario){
        this.propietario = propietario;
        if (propietario != null && propietario.getComputadora() != this){
            propietario.setComputadora (this);
        }
    }

    public String getMarca() {
```

EjercicioRelacion5.java [-/A] × | Computadora.java [-/A] × | PlacaMadre.java [-/A] × | Propietario.java [-/A] ×

Source | History

```java
/**
 *
 * @author HP
 */
public class PlacaMadre {
    private String modelo;
    private String chipset;

    public PlacaMadre(String modelo, String chipset) {
        this.modelo = modelo;
        this.chipset = chipset;
    }

    public String getModelo() {
        return modelo;
    }

    public String getChipset() {
        return chipset;
    }


}
```

EjercicioRelacion5.java [-/A] × | Computadora.java [-/A] × | PlacaMadre.java [-/A] × | Propietario.java

Source | History

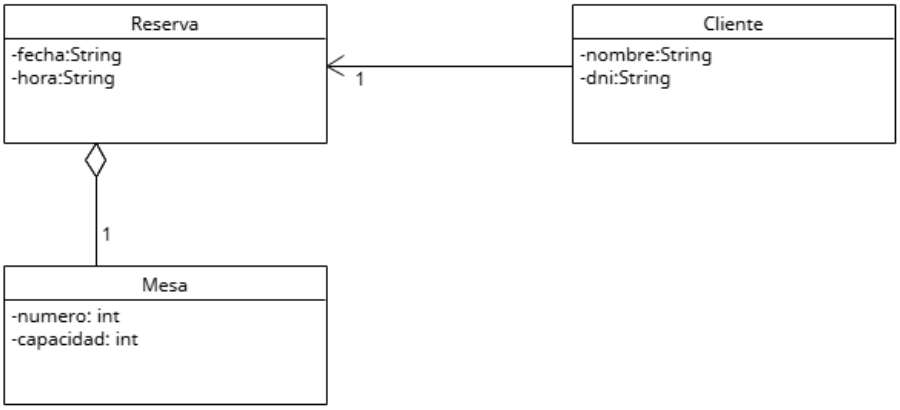
```java
 * @author HP
 */
public class Propietario {
    private String nombre;
    private String dni;
    private Computadora computadora;//referencia a la computadora asociada

    public Propietario(String nombre, String dni) {
        this.nombre = nombre;
        this.dni = dni;
    }

    public void setComputadora(Computadora computadora){
        this.computadora= computadora;
        if (computadora !=null && computadora.getPropietario() != this){
            computadora.setPropietario(this);
        }
    }

    public String getNombre() {
        return nombre;
    }

    public String getDni() {
        return dni;
    }

    public Computadora getComputadora() {
```

6. Reserva - Cliente - Mesa

a. Asociación unidireccional: Reserva → Cliente
b. Agregación: Reserva → Mesa

Clases y atributos:

i. Reserva: fecha, hora
ii. Cliente: nombre, teléfono (en vez de poner teléfono use dni)
iii. Mesa: numero, capacidad




EjercicioRelacion6.java [-/A] × Reserva.java [-/A] × Mesa.java [-/A] × Cliente.java [-/A] ×

Source History

```java
/**
 *
 * @author HP
 */
public class EjercicioRelacion6 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Mesa mesa =new Mesa ("1","5");
        Reserva reserva= new Reserva ("25/10/2025 ", " 22.00", mesa);
        Cliente cliente= new Cliente("Oscar Longo ","065859");
        cliente.setReserva(reserva);
        cliente.mostrarReserva();
        reserva.mostrarMesa();
    }

}
```

Output ×

HP - C:\Users\HP × EjercicioRelacion6 (run) ×

```
run:
El cliente Oscar Longo numero de DNI:065859 tiene una reserva el dia 25/10/2025 a las: 22.00 horas
en la fecha: 25/10/2025 hora:  22.00tiene la mesa 1con capacidad: 5
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
/**
 *
 * @author HP
 */
public class Reserva {
    private String fecha;
    private String hora;
    private Mesa mesa;//agregacion

    public Reserva(String fecha, String hora, Mesa mesa) {
        this.fecha = fecha;
        this.hora = hora;
        this.mesa = mesa;
    }

    public String getFecha() {
        return fecha;
    }

    public String getHora() {
        return hora;
    }
    public void mostrarMesa(){
        System.out.println("en la fecha: " + fecha + "hora: "+ hora + "tiene la mesa " + mesa.getNumero(
    }
```

```java
/
blic class Mesa {
  private String numero;
  private String capacidad;

  public Mesa(String numero, String capacidad) {
      this.numero = numero;
      this.capacidad = capacidad;
  }

  public String getNumero() {
      return numero;
  }

  public void setNumero(String numero) {
      this.numero = numero;
  }

  public String getCapacidad() {
      return capacidad;
  }

  public void setCapacidad(String capacidad) {
      this.capacidad = capacidad;
  }
```

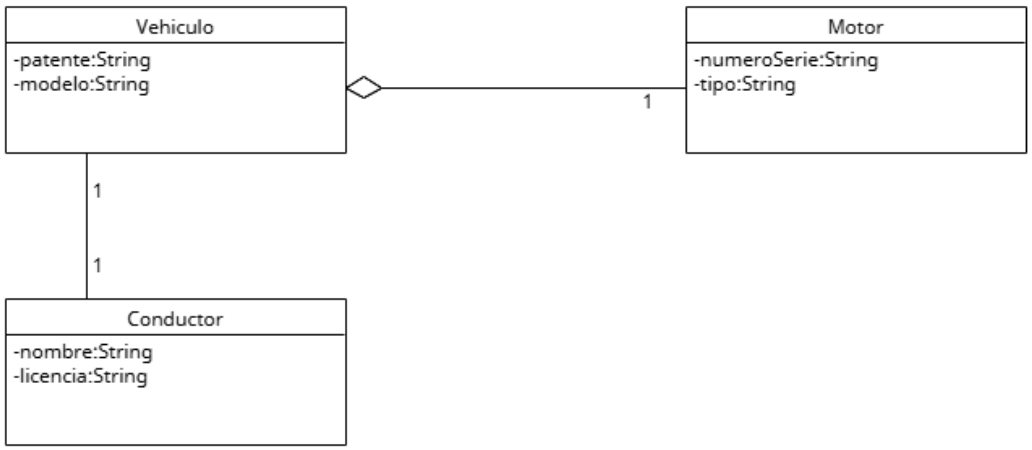
```java
public class Cliente {
    private String nombre;
    private String dni;
    private Reserva reserva; // asociacion 1 a 1

    public Cliente(String nombre, String dni) {
        this.nombre = nombre;
        this.dni = dni;
    }

    public void setReserva(Reserva reserva){
        this.reserva = reserva;
    }
    public  Reserva getReserva(){
        return reserva;
    }

    public void mostrarReserva(){
        if (reserva != null ){
            System.out.println("El cliente " + nombre + "numero de DNI:" + dni + " tiene una reserva

        }else {
            System.out.println("El cliente " + nombre + "numero de DNI:" + dni + "no posee reserva");
```

7. Vehículo - Motor - Conductor

a. Agregación: Vehículo → Motor
b. Asociación bidireccional: Vehículo ↔ Conductor

Clases y atributos:

i. Vehículo: patente, modelo
ii. Motor: tipo, numeroSerie
iii. Conductor: nombre, licencia



EjercicioRelacion7.java [-/A] × Conductor.java [-/A] × Vehiculo.java [-/A] × Motor.java [-/A] ×

ource History

```java
 * @author HP
 */
public class EjercicioRelacion7 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Motor motor =new Motor (" v8 ",  " naftero ");
        Vehiculo vehiculo = new Vehiculo (" AF859HC " , " Cadilac ", motor);
        Conductor conductor =new Conductor("Osvaldo Rios" ," A273635");

        vehiculo.mostrarMotor();
        conductor.setVehiculo(vehiculo);

         System.out.println("El conductor " + conductor.getNombre() + " licencia " + condu
    }
}
```

utput ×

HP - C:\Users\HP × EjercicioRelacion7 (run) ×

```
run:
 La patente:  AF859HC  modelo  Cadilac  tiene un motor con numero de serie v8  tipo  naftero
El conductor Osvaldo Rios licencia  A273635
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
 */
public class Conductor {
    private String nombre;
    private String licencia;
    private Vehiculo vehiculo;//referencia al vehiculo asociado

    public Conductor(String nombre, String licencia) {
        this.nombre = nombre;
        this.licencia = licencia;
    }

    public void setVehiculo(Vehiculo vehiculo){
        this.vehiculo= vehiculo;
        if(vehiculo !=null && vehiculo.getConductor() !=this){
            vehiculo.setConductor(this);
        }

    }
    public Vehiculo getVehiculo(){
```

```java
public class Vehiculo {
    private  String patente;
    private  String modelo;
    private  Motor motor;//agregacion
    private Conductor conductor;//Asocicaion

    public Vehiculo(String patente, String modelo, Motor motor) {
        this.patente = patente;
        this.modelo = modelo;
        this.motor = motor;
    }

    public void mostrarMotor(){
        if (motor!=null){
            System.out.println(" La patente: " + patente + " modelo " +  modelo +
                    " tiene un motor con numero de serie"  + motor.getNumeroSerie()
                    + " tipo " + motor.getTipo());
        }else{
            System.out.println(" La patente: " + patente + " modelo " +  modelo + " no tiene motor.");
```

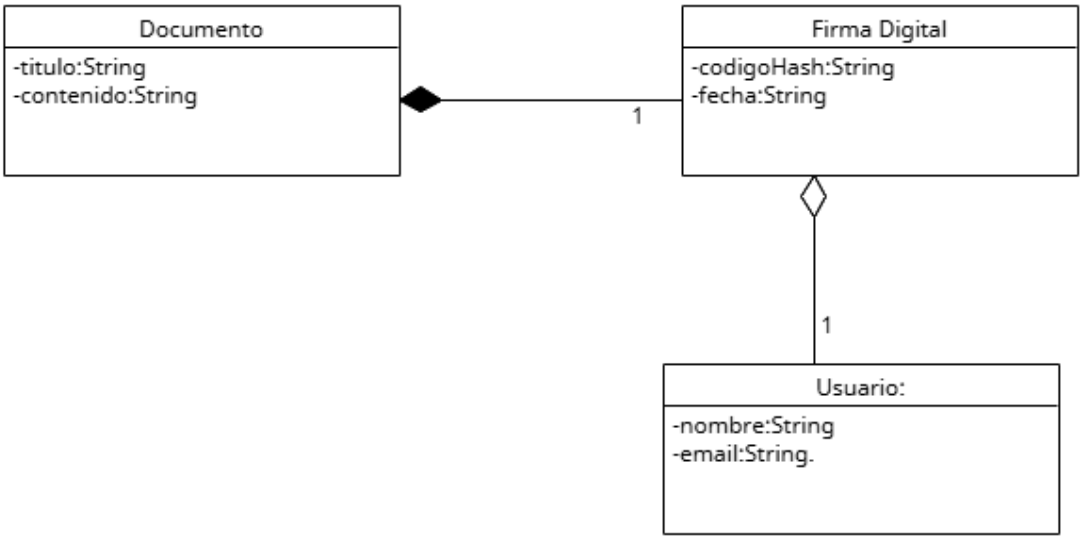
```java
public class Motor {
    private String numeroSerie;
    private String tipo;
    private Vehiculo vehiculo;

    public Motor(String numeroSerie, String tipo) {
        this.numeroSerie = numeroSerie;
        this.tipo = tipo;

    }

    public String getNumeroSerie() {
        return numeroSerie;
    }

    public void setNumeroSerie(String numeroSerie) {
        this.numeroSerie = numeroSerie;
    }
```

8. Documento - FirmaDigital - Usuario

a. Composición: Documento → FirmaDigital
b. Agregación: FirmaDigital → Usuario

Clases y atributos:

i. Documento: titulo, contenido
ii. FirmaDigital: codigoHash, fecha
iii. Usuario: nombre, email




EjercicioRelacion8.java [-/A] × | Documento.java [-/A] × | FirmaDigital.java [-/A] × | Usuario.java [-/A] ×

Source | History

```java
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {

        Usuario cliente=new Usuario ("Juan Perez", "jperez123@gmail.com");

        Documento documento1= new Documento("Documento", "prestamo", "cod254185", "25/10/2020");
        documento1.getFirmaDigital().setUsuario(cliente);

        documento1.mostrarFirmaDigital();
        documento1.mostrarUsuario();


    }

}
```

utput ×

HP - C:\Users\HP × | EjercicioRelacion8 (run) ×

```
run:
El Documento con el contenido prestamo corresponde al codigo/fecha: cod25418525/10/2020
El codigo: cod254185correspondiente a la fecha: 25/10/2020corresponde al usuario:Juan Perezmail:jperez123@gmail.com
BUILD SUCCESSFUL (total time: 0 seconds)
```

EjercicioRelacion8.java [-/A] × | Documento.java [-/A] × | FirmaDigital.java [-/A] × | Usuario.java [-/A] ×

Source | History

```java
*/
public class Documento {
    private String titulo;
    private String contenido;
    private FirmaDigital firmaDigital;// composicion 1 a 1

    public Documento(String titulo, String contenido, String codigoHash, String fecha){
        this.titulo = titulo;
        this.contenido = contenido;
        this.firmaDigital= new FirmaDigital (codigoHash, fecha); //se crea internamnete


    }
    public void mostrarFirmaDigital(){
        System.out.println("El " + titulo + " con el contenido " + contenido +
                " corresponde al codigo/fecha: "
                + firmaDigital.getCodigoHash() + firmaDigital.getFecha());
    }

    public void mostrarUsuario() {
        firmaDigital.mostrarUsuario();
```

EjercicioRelacion8.java [-/A] × | Documento.java [-/A] × | FirmaDigital.java [-/A] × | Usuario.java [-/A] ×

Source | History

```java


    public void setUsuario(Usuario usuario) {
        this.usuario = usuario;
    }


    public void mostrarUsuario(){
        if(usuario !=null){


        System.out.println("El codigo: " + codigoHash +
                "correspondiente a la fecha: " + fecha +
                "corresponde al usuario:"+ usuario.getNombre ()+ "mail:"+ usuario.getEmail
        }else{
            System.out.println("El codigo: " + codigoHash +
                "correspondiente a la fecha: " + fecha +
                "no corresponde a ningun usuario");
        }
    }
```

Output ×

HP - C:\Users\HP × | EjercicioRelacion8 (run) ×

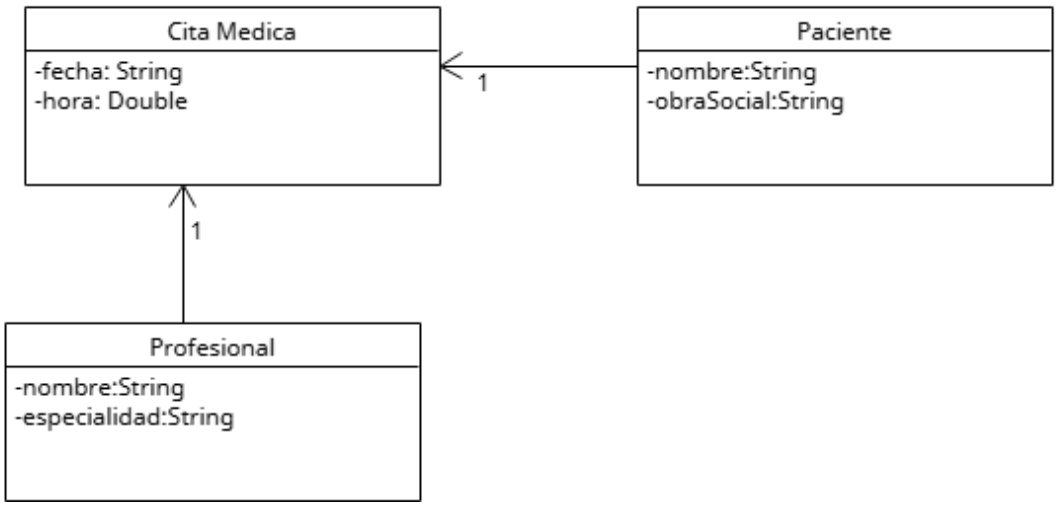
```java
/**
 *
 * @author HP
 */
public class Usuario {
    private String nombre;
    private String email;

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEmail() {
        return email;
    }
```

9. CitaMédica - Paciente - Profesional

a. Asociación unidireccional: CitaMédica → Paciente,
b. Asociación unidirecciona: CitaMédica → Profesional

Clases y atributos:

i. CitaMédica: fecha, hora
ii. Paciente: nombre, obraSocial
iii. Profesional: nombre, especialidad




EjercicioRelacion9.java [-/A] × Paciente.java [-/A] × CitaMedica.java [-/A] × Profesional.java [-/A] ×

History

```java
 * @author HP
 */
public class EjercicioRelacion9 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {

        Paciente paciente1=new Paciente ("Ricardo Jaime ", "Osde");
        Profesional profesional1= new Profesional ("Laura Lopez", "Odontologa");

        CitaMedica cita =new CitaMedica("24/08/2024", "8.00", paciente1, profesional1);
        cita.mostrarCita();

    }
}
```

HP - C:\Users\HP × EjercicioRelacion9 (run) ×

```
run:
Cita Medica: 24/08/2024 a las 8.00
El paciente: Ricardo Jaime  con obra social: Osde
El profesional: Laura Lopez especialidad Odontologa
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
* @author HP
*/
public class Paciente {
    private String nombre;
    private String obraSocial;

    public Paciente(String nombre, String obraSocial) {
        this.nombre = nombre;
        this.obraSocial = obraSocial;
    }

    public String getNombre() {
        return nombre;
        }

    public String getObraSocial() {
        return obraSocial;
    }
```

EjercicioRelacion9.java [-/A] × | Paciente.java [-/A] × | CitaMedica.java [-/A] × | Profesional.java [-/A] ×

History

```java
    public CitaMedica(String fecha, String hora,Paciente paciente, Profesional profesional)
        this.fecha = fecha;
        this.hora = hora;
        this.paciente= paciente;
        this.profesional=profesional;

    }

    public void mostrarCita() {
    System.out.println("Cita Medica: " + fecha + " a las " + hora);
    if (paciente != null) {
        paciente.mostrarDatos();
    } else {
        System.out.println("Paciente no asignado.");
    }

    if (profesional != null) {
        profesional.mostrarDatos();
    } else {
        System.out.println("Profesional no asignado.");
```

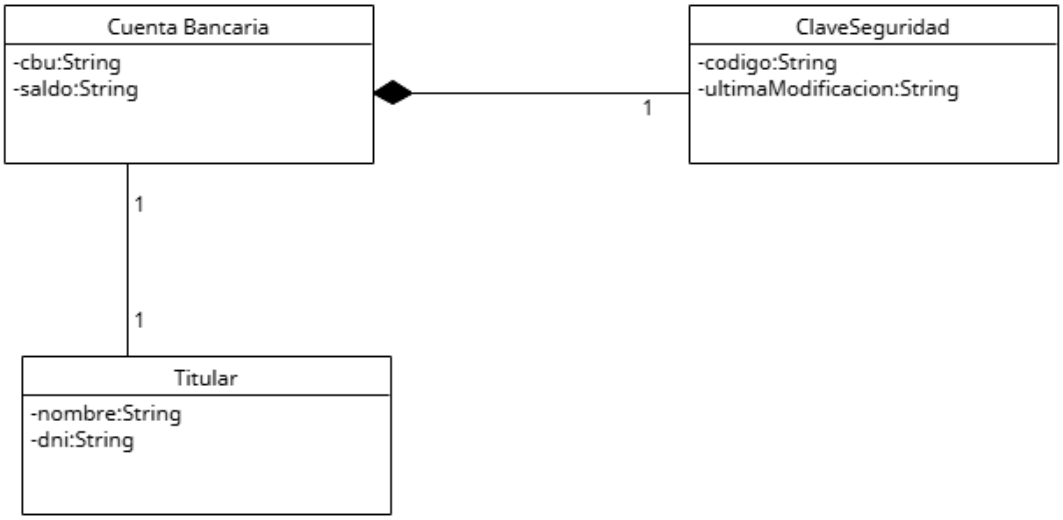
```java
 * @author HP
 */
public class Profesional {
    private String nombre;
    private String especialidad;


    public Profesional(String nombre, String especialidad) {
        this.nombre = nombre;
        this.especialidad = especialidad;
    }




    public void mostrarDatos(){
       System.out.println("El profesional: " + nombre + " especialidad " + especialidad );
```

10. CuentaBancaria - ClaveSeguridad - Titular

a. Composición: CuentaBancaria → ClaveSeguridad
b. Asociación bidireccional: CuentaBancaria ↔ Titular

Clases y atributos:

i. CuentaBancaria: cbu, saldo
ii. ClaveSeguridad: codigo, ultimaModificacion
iii. Titular: nombre, dni.



```java
package ejercicioRelacion10;

/**
 *
 * @author HP
 */
public class EjercicioRelacion10 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Titular titular1= new Titular("Eliana Gomez", "20658963");
        CuentaBancaria cuentaBancaria= new CuentaBancaria("007209654874512", "200", "2547", "25-0

        titular1.setCuentaBancaria(cuentaBancaria);
        cuentaBancaria.mostrarDatos();

    }
```

utput ×

HP - C:\Users\HP × | EjercicioRelacion10 (run) ×

```
run:
El cbu: 007209654874512tiene un saldo: 200 posee un codigo:2547 y se modifico por ultima vez: 25-08-1998
Titular:Eliana Gomezdni 20658963
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
 */
public class ClaveSeguridad {
    private String codigo;
    private String ultimaModificacion;

    public ClaveSeguridad(String codigo, String ultimaModificacion
        this.codigo = codigo;
        this.ultimaModificacion = ultimaModificacion;
    }

    public String getCodigo() {
        return codigo;
    }

    public String getUltimaModificacion() {
        return ultimaModificacion;
```

```java
EjercicioRelacion10.java [-/A] × | ClaveSeguridad.java [-/A] × | CuentaBancaria.java [-/A] × | Titular.java [-/A] ×
Source | History
*/
public class CuentaBancaria {
    private String cbu;
    private String saldo;
    private ClaveSeguridad claveSeguridad;//composicion
    private Titular titular;//asociacion bidireccional


    public CuentaBancaria(String cbu, String saldo, String codigo, String ultimaModificacion)
        this.cbu = cbu;
        this.saldo = saldo;
        this.claveSeguridad =  new ClaveSeguridad(codigo, ultimaModificacion);

    }

    public void setTitular(Titular titular) {
        this.titular= titular;
        if (titular !=null && titular.getCuentaBancaria() != this){
        titular.setCuentaBancaria(this);
        }
```

```java
public class Titular {
    private String nombre;
    private String dni;
    private CuentaBancaria cuentaBancaria;

    public Titular(String nombre, String dni) {
        this.nombre = nombre;
        this.dni = dni;
    }
    public void setCuentaBancaria(CuentaBancaria cuentaBancaria) {
    if (this.cuentaBancaria != cuentaBancaria) {
        this.cuentaBancaria = cuentaBancaria;
        if (cuentaBancaria != null && cuentaBancaria.getTitular() != this)
            cuentaBancaria.setTitular(this);
        }
```

DEPENDENCIA DE USO

La clase usa otra como parámetro de un método, pero no la guarda como atributo.

Ejercicios de Dependencia de Uso

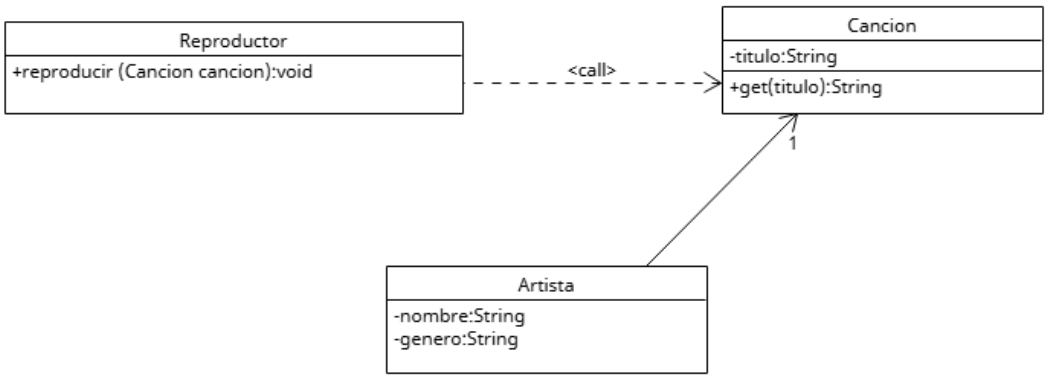
11. Reproductor - Canción - Artista

a. Asociación unidireccional: Canción → Artista
b. Dependencia de uso: Reproductor.reproducir(Cancion)

Clases y atributos:

i. Canción: titulo.
ii. Artista: nombre, genero.

iii. Reproductor->método: void reproducir(Cancion cancion)




```
 * @author HP
 */
public class EjercicioDependenciaUso11 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Cancion cancion= new Cancion ("El favor");
        Artista artista=new Artista("Las pastillas del abuelo" ,"rock");
        Reproductor reproducir= new Reproductor ();

        artista.setCancion(cancion);
        reproducir.reproducir(cancion);

        artista.mostrarCancion();
    }

}
```

HP - C:\Users\HP × EjercicioDependenciaUso11 (run) ×

```
run:
El favor
La cancion de Las pastillas del abuelo  pertenece al genero rock se llama: El favor
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
package ejerciciodependenciausoll;

/**
 *
 * @author HP
 */
public class Reproductor {
    public void reproducir (Cancion cancion){
        System.out.println(cancion.getTitulo());
    }
```

```java
 */
public class Cancion {
    private String titulo;

    public Cancion(String titulo) {
        this.titulo = titulo;
    }

    public String getTitulo() {
        return titulo;
```

EjercicioDependenciaUso11.java [-/A] × | Reproductor.java [-/A] × | Cancion.java [-/A] × | Artista.java [-/A] ×

Source | History

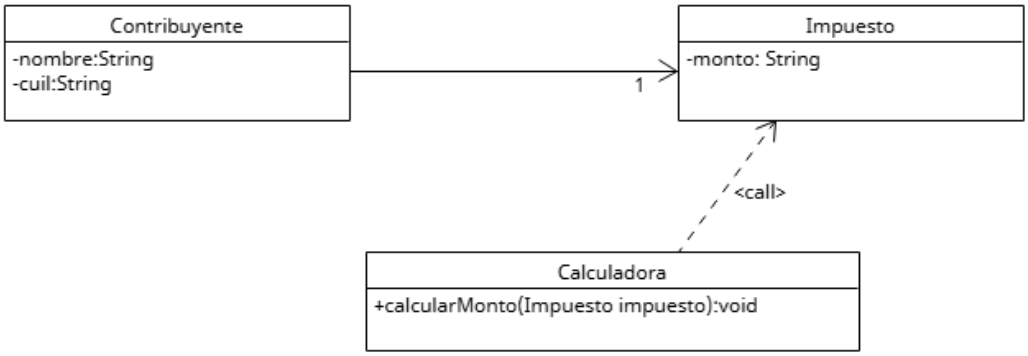
```java
 */
public class Artista {
    private String nombre;
    private String genero;
    private Cancion cancion;// asociacion lal

    public Artista(String nombre, String genero) {
        this.nombre = nombre;
        this.genero = genero;
    }

    public void setCancion (Cancion cancion){
        this.cancion= cancion;

    }
    public void mostrarCancion (){
        if (cancion !=null){
            System.out.println("La cancion de " + nombre +
                    "  pertenece al genero " + genero + " se llama: " + cancion.getTitulo());
        } else{
            System.out.println("El cantante " + nombre +
```

12. Impuesto - Contribuyente - Calculadora

a. Asociación unidireccional: Impuesto → Contribuyente
b. Dependencia de uso: Calculadora.calcular(Impuesto)

Clases y atributos:

i. Impuesto: monto.
ii. Contribuyente: nombre, cuil.
iii. Calculadora->método: void calcular(Impuesto impuesto)




iv.

```
 * @author HP
 */
public class EjercicioDepUso12 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Impuesto impuesto=new Impuesto("25000");
        Contribuyente contribuyente= new Contribuyente("Carlos Noir", "23205698567");
        Calculadora calcularMonto=new Calculadora();

        contribuyente.setImpuesto(impuesto);
        contribuyente.mostrarImpuesto();
    }

}
```

```
- C:\Users\HP ×    EjercicioDepUso12 (run) ×
run:
l cliente: Carlos Noircuil 23205698567abona:25000
```

```
package ejerciciodepuso12;

/**
 *
 * @author HP
 */
public class Calculadora {
    public void calcularMonto(Impuesto impuesto){
        System.out.println(impuesto.getMonto());
    }
```

```java
 * @author HP
 */
public class Impuesto {
    private String monto;

    public Impuesto(String monto) {
        this.monto = monto;
    }

    public String getMonto() {
        return monto;
```

```java
 */
public class Contribuyente {
    private String nombre;
    private String cuil;
    private Impuesto impuesto;

    public Contribuyente(String nombre, String cuil) {
        this.nombre = nombre;
        this.cuil = cuil;
    }

    public void setImpuesto(Impuesto impuesto) {
        this.impuesto = impuesto;
    }

    public void mostrarImpuesto (){
        if (impuesto !=null){
            System.out.println("El cliente: " + nombre + "cuil "
                    + cuil + "abona:" + impuesto.getMonto());
        }else{
```

DEPENDENCIA DE CREACIÓN

La clase crea otra dentro de un método, pero no la conserva como atributo..

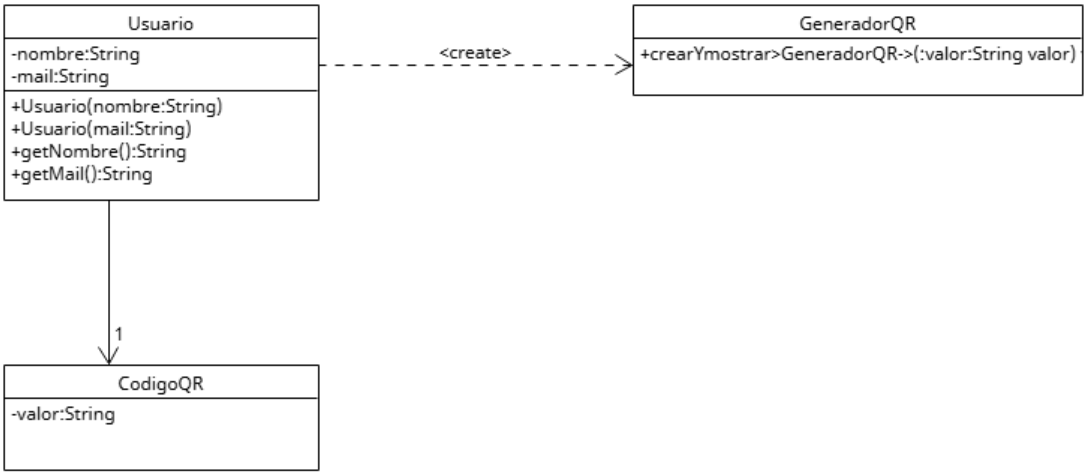
Ejercicios de Dependencia de Creación

13. GeneradorQR - Usuario - CódigoQR

a. Asociación unidireccional: CódigoQR → Usuario
b. Dependencia de creación: GeneradorQR.generar(String, Usuario)

Clases y atributos:

i. CodigoQR: valor.
ii. Usuario: nombre, email.
iii. GeneradorQR->método: void generar(String valor, Usuario usuario)




EjercicioDepCreacion13.java [-/A] × | CodigoQR.java [-/A] × | Usuario.java [-/A] × | GeneradorQR.java [-/A] ×

Source | History

```java
10  */
11  public class EjercicioDepCreacion13 {
12
13      /**
14       * @param args the command line arguments
15       */
16      public static void main(String[] args) {
17          CodigoQR codigoQr= new CodigoQR ("A4754JILO96");
18          Usuario usuario = new Usuario ("Domingo Lopez", " dlopezmingo@gmail.com");
19          GeneradorQR generadorQr =new GeneradorQR();
20
21          generadorQr.crearYMostrar(usuario.getNombre());
22          usuario.setCodigoQR(codigoQr);
23          usuario.mostrarCodigoQR();
24
25
26
27      }
28
29  }
30
```

Output ×

HP - C:\Users\HP × | EjercicioDepCreacion13 (run) ×

```
run:
El usuario se llama: Domingo Lopez
Cliente: Domingo Lopez ,mail:  dlopezmingo@gmail.com tiene asociado el siguiente QR:A4754JILO96
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
/**
 *
 * @author HP
 */
public class CodigoQR {
    private String valor;

    public CodigoQR(String valor) {
        this.valor = valor;
    }

    public String getValor() {
        return valor;
    }
```

EjercicioDepCreacion13.java [-/A] × | CodigoQR.java [-/A] × | Usuario.java [-/A] × | GeneradorQR.java [-/A] ×

Source | History

```java
public class Usuario {
    private String nombre;
    private String mail;
    private CodigoQR codigoQr;

    public Usuario(String nombre, String mail) {
        this.nombre = nombre;
        this.mail = mail;
    }

    public void setCodigoQR (CodigoQR codigoQr){
        this.codigoQr= codigoQr;

    }

    public void mostrarCodigoQR(){
        if (codigoQr != null ) {
            System.out.println("Cliente: " + nombre + " ,mail: " + mail + " tiene asociado el siguiente
        }else{
            System.out.println("Cliente: " + nombre + "mail: "+ mail + " no posee codigo QR");
```

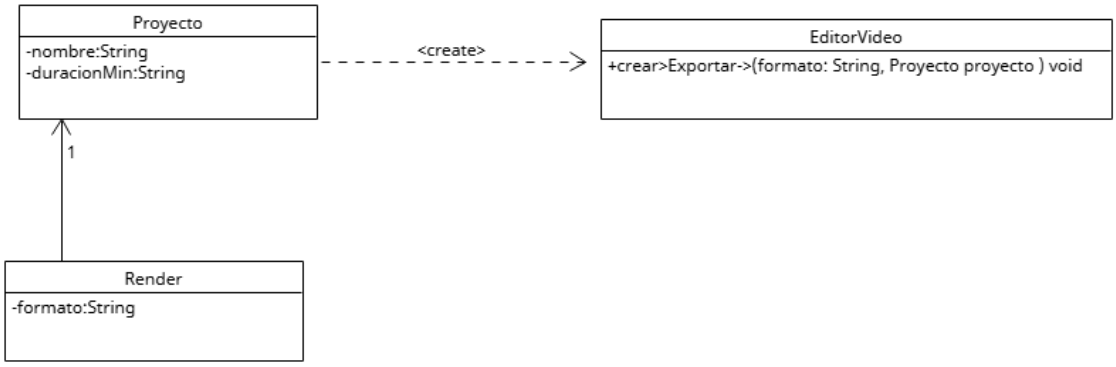
```java
 *
 * @author HP
 */
public class GeneradorQR {
    public void crearYMostrar(String nombre){



        System.out.println("El usuario se llama: " + nombre);

    }
}
```

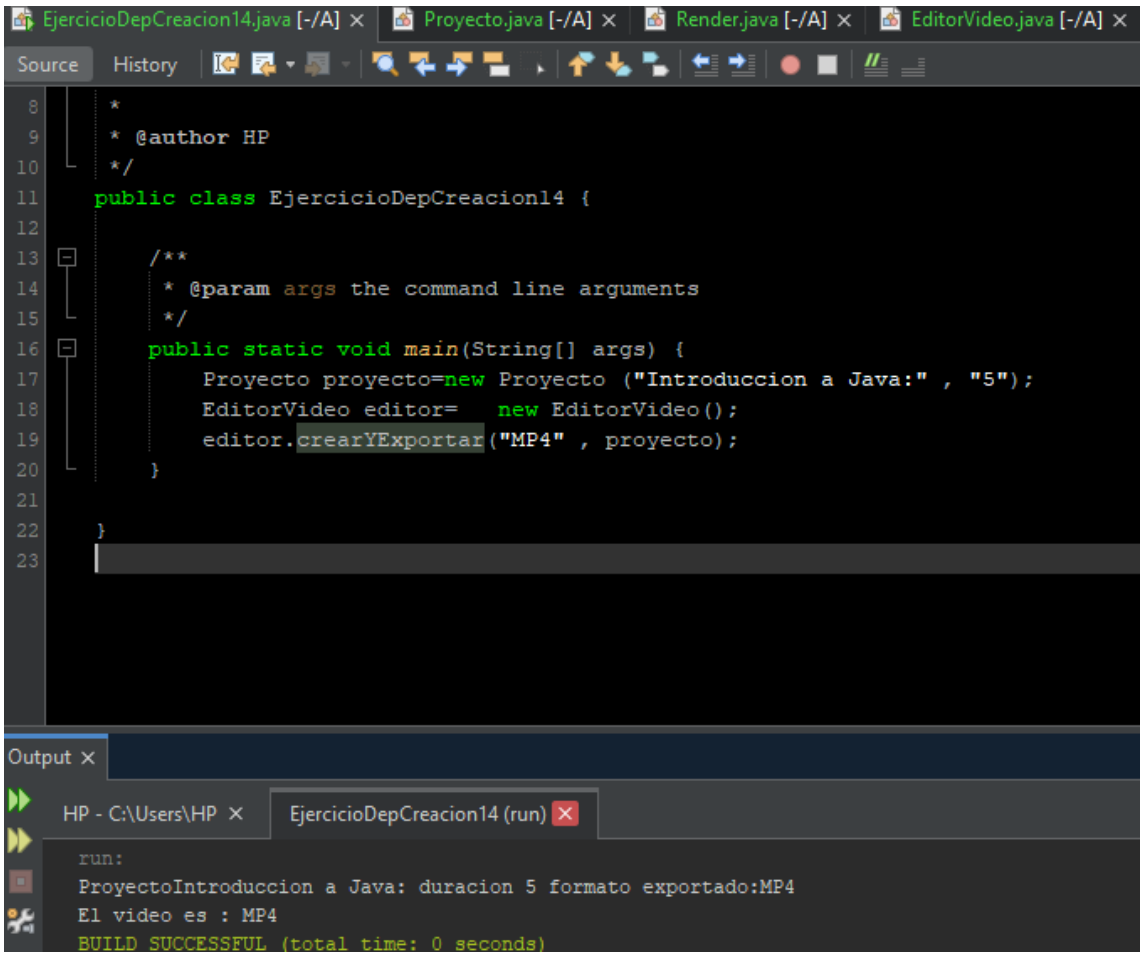
14. EditorVideo - Proyecto - Render

a. Asociación unidireccional: Render → Proyecto
b. Dependencia de creación: EditorVideo.exportar(String, Proyecto)

c. Clases y atributos:

i. Render: formato.
ii. Proyecto: nombre, duracionMin.
iii. EditorVideo->método: void exportar(String formato, Proyecto proyecto)



EjercicioDepCreacion14.java [-/A] × | Proyecto.java [-/A] × | Render.java [-/A] × | EditorVid

Source | History

```
7
8
9   author HP
10
11  ic class Proyecto {
    private String nombre;
    private String duracionMin;
14  private  Render render;
15
16  public Proyecto(String nombre, String duracionMin) {
17      this.nombre = nombre;
18      this.duracionMin = duracionMin;
19  }
20
21  public void setRender (Render render){
22      this.render = render;
23
24  }
25
26  public void mostrarRender(){
```

EjercicioDepCreacion14.java [-/A] × | Proyecto.java [-/A] × | Render.java [-/A] × | EditorVideo.java [-/A] ×

Source | History

```
4   */
5   package ejerciciodepcreacion14;
6
7   /**
8    *
9    * @author HP
10   */
11  public class Render {
        private String formato;
13
14      public Render(String formato) {
15          this.formato = formato;
16      }
17
18      public String getFormato() {
19          return formato;
```

```
 *
 * @author HP
 */
public class EditorVideo {
    public void crearYExportar(String formato, Proyecto proyecto){
        Render render = new Render(formato);
        proyecto.setRender(render);
        proyecto.mostrarRender();



        System.out.println("El video es : " + formato);
```

CONCLUSIONES ESPERADAS

- ● Diferenciar claramente los tipos de relaciones entre clases (asociación,
- agregación, composición).
- ● Representar las relaciones con la dirección adecuada en diagramas UML.
- ● Comprender e implementar dependencias de uso y de creación.
- ● Aplicar relaciones 1 a 1 en el diseño e implementación de clases en Java.
- ● Reforzar el análisis de modelos orientados a objetos y la capacidad de
- abstracción.